
AUTOMATED DISRUPTION LABELING OF DIII-D SHOTS USING A 1D CONVOLUTIONAL NEURAL NETWORK AND TEMPORAL FILTERING OF PLASMA CURRENT

Jason Jain, Jesse A. Rodríguez

School of Mechanical, Industrial, and Manufacturing Engineering
Oregon State University
Corvallis, OR 97331, USA
jainsp@oregonstate.edu, jesse.rodriguez@oregonstate.edu

August 5, 2026

ABSTRACT

Experimental Tokamak fusion facilities like DIII-D frequently experience disruption events that result in a plasma current drop to zero. The existence of a computationally inexpensive model for identifying disruptive shots and accurately identifying time of disruption can assist the scientific community by auto-labeling shots for further analysis. We took a labeled plasma disruption dataset from the DIII-D Tokamak reactor, utilizing shot numbers 125500-186000 (nearly 40,000 total plasma shots) to develop a new disruption labeling model that can generate automated disruption labeling using only plasma current time series data. We train a CNN (Convolutional Neural Network) to perform boolean disruption/not-disruption prediction and achieve a 99% recall and 91% precision on our test dataset. We additionally develop a classical convolutional filter heuristic that is capable of accurately identifying time of disruption with a median of 0 and standard deviation of 1.1 ms on our test dataset.

1 Introduction

A Tokamak is a torus-shaped magnetic confinement fusion reactor and is a leading candidate to produce controlled thermonuclear fusion power [1]. A prominent challenge facing Tokamak reactors is the issue of disruption events caused by turbulent instability within the plasma. Such disruption events cause the internal reactor current ($I(t)$) to drop to zero and can potentially cause damage to the interior of the reactor. Abnormalities such as plasma disruptions are a prominent challenge for upcoming flagship fusion experiments such as The International Thermonuclear Experimental Reactor (ITER) [2, 1].

Decades of Tokamak experiments have created a large corpus of fusion data, leading to a growing body of literature utilizing machine learning for problems infeasible to address with classical methods, including disruption prediction [3]. A variety of classical machine learning algorithms have demonstrated promise; support vector ma-

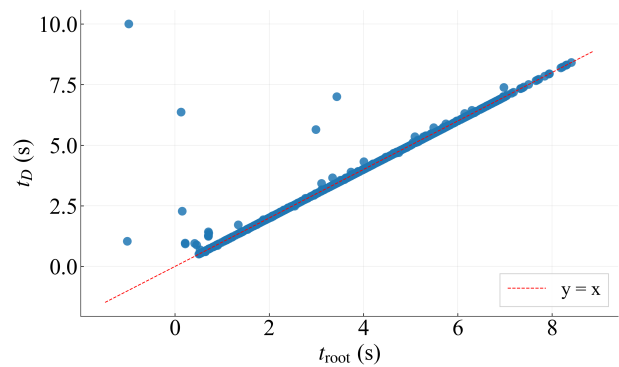


Figure 1: Predicted disruption time Eq (7) vs real disruption time for 889 test holdout shots.

chines, fuzzy logic, generative topographic mapping, and artificial neural networks have all been studied on a broad range of Tokamaks across the United States, India, China,

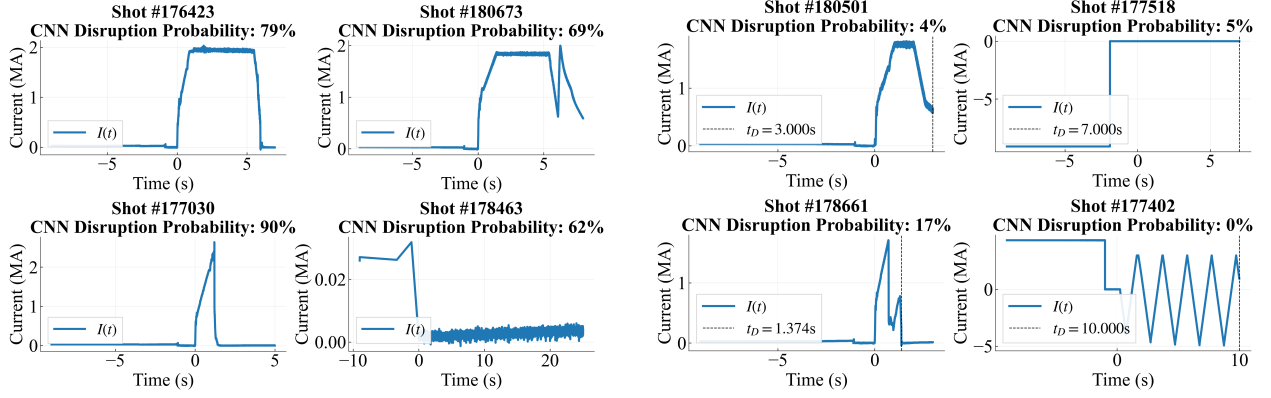


Figure 2: On the left, four selected non-disruptive shots incorrectly identified as disruptive by the CNN. Shots 176423 and 177030 feature a drop resembling a disruption, thereby confusing our model, while shot 178463 appears to be garbage data based off the current peaking at ≈ 0.02 MA. On the right, four selected disruptive shots incorrectly identified as non-disruptive by the CNN. Each shot lacks the characteristic instant drop to zero that typically identifies a disruption. Shots 180501 and 177402 do not present an evident disruption and seem erroneously labeled.

and Europe [4]. In the deep learning space, sequence based models such as Long-Short Term Memory Networks (LSTMs) show promise for dynamic time-series forecasting of plasma disruptions [5]. A variety of approaches have also been proposed for knowledge transfer from existing devices to new ones, using hybrid architectures [6] and transfer learning [7].

Such models require large labeled datasets for training and testing, and would greatly benefit from an algorithm capable of auto-labeling novel fusion shots. We posit such an algorithm in this paper, seeing promising performance on 889 disruptive shots in our test holdout dataset (Figure 1). We hope that our work can enable the scientific community to generate larger labeled datasets for training the next generation of disruption prediction models.

2 Neural Net Classifier

A large fraction of the data produced by long-running Tokamak fusion facilities such as the DIII-D reactor operated by General Atomics has already been labeled for training deep learning models. We were provided 39438 disruption-labeled shot files from the DIII-D team, of which 13429 were disruptive shots with a given time of disruption.

Each shot in our dataset sampled reactor current at 4 kHz, giving us a discrete $1 \times m$ current vector $\mathbf{I}$ where the m th component is current at the m th timestep.

$$\mathbf{I}_p = [I_1 \quad \dots \quad I_m]^T \quad (1)$$

We used an 80/10/10 train/test/dev split, which left us with 31542 shots in the training set. To prepare our data for the neural net classifier, we tested three different normalization methods, including per-shot min-max unit scale normalization, per-shot Mean-Variance (meanvar) normalization,

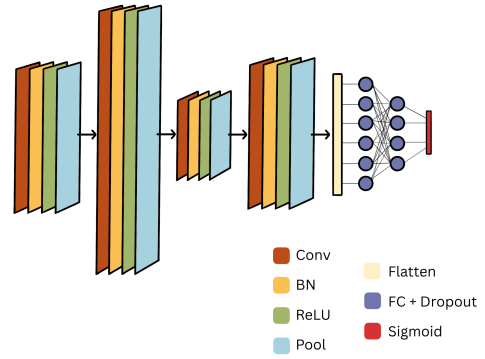


Figure 3: Model architecture for our disruption labeling Convolutional Neural Network. We have four convolutional layers and two fully connected layers, and utilize max pooling, batch normalization, and dropout, along with a ReLU activation function. The model input is normalized current sampled at 4 kHz and the output is disruption probability for the shot.

and dataset-wide meanvar normalization. We found most success with the dataset-wide meanvar approach. Letting the current mean and standard deviation across all shots be μ and σ , we computed a normalized shot tensor $\mathbf{I}_N$ for the arbitrary i th shot with N_i timesteps. If the longest shot has N timesteps, we pad the i th shot with $N - N_i$ trailing zeroes.

$$\mathbf{I}_{p,N} = \begin{bmatrix} \frac{1}{\sigma}(\mathbf{I}_p - \mu) \\ \mathbf{0}_{(N-N_i) \times 1} \end{bmatrix} \quad (2)$$

The normalized data was then fed into a Convolutional Neural Network (see Figure 3) consisting of four convo-

lutional layers with stride 1, each with batch normalization [8], a Rectified Linear Unit (ReLU) activation function [9], and a final max pool layer [10]. The convolution outputs were then fed into two fully connected layers with batch normalization, ReLU, and a dropout layer with a tunable dropout rate dropout hyperparameter.

The CNN was programmed in PyTorch [11] and trained using the Adam (Adaptive Moment Estimation) optimizer [12] with weight decay and a learning rate tuned per-epoch with PyTorch’s `lr_scheduler.ReduceLROnPlateau` in "min" mode [13]. We utilized PyTorch’s `torch.nn.BCEWithLogitsLoss` with configurable positive weight and gradient clipping, and stopped training when enough consecutive epochs showed no improvement.

We tuned our hyperparameters with Bayesian Optimization using the ExpectedImprovement acquisition function with $\xi = 0.05$ [14]. We additionally inserted random samples every 5 trials to escape local minima. Our scoring strategy was a weighted F_β [15], where parameter β is the weighted harmonic mean of precision and recall (Eq (3)). We chose $\beta = 1.5$ to prioritize recall (R) over precision (P) and minimize false negatives, rejecting trials below a precision floor of 90%.

$$F_\beta = \frac{PR(1 + \beta^2)}{P\beta^2 + R} \quad (3)$$

We tuned model and architecture hyperparameters for 100+ trials on the Polaris supercomputer in Argonne National Lab, with each trial sampling fresh hyperparameters and training a new model for up to 200 epochs. Increasing the decision threshold gave us higher recall at the cost of precision, as seen in Figure 4. With a decision threshold of 50%, our best model achieved a precision of 90.52% and recall of 98.84% on the test set, along with an F1 score of 0.9450 and F2 score of 0.9706. The tuned hyperparameters are presented in Table 1.

Table 1: Tuned Hyperparameters

Hyperparameter	Value
Learning Rate	0.0005
Number of Epochs	200
Dropout Rate	0.35
Weight Decay	0.0001
Training Batch Size	128
Gradient Clip	1.0
Learning Rate Scheduler Factor	0.5
Learning Rate Patience	5
Early Stopping Patience	10
Positive Weight	3.0

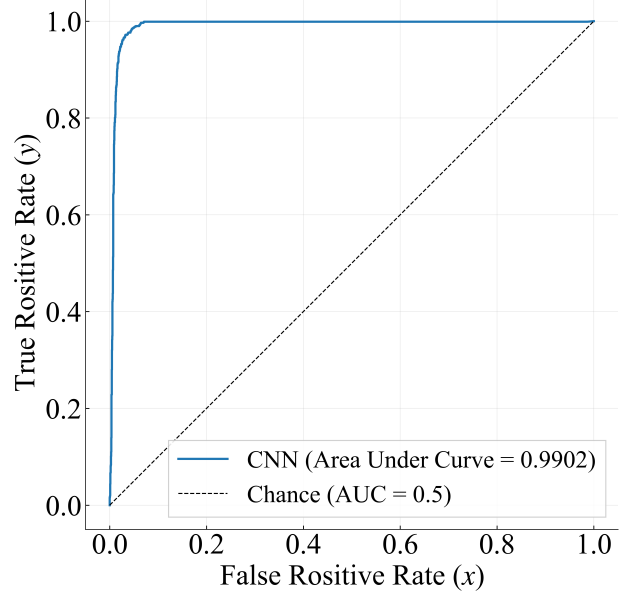


Figure 4: The ROC curve for our disruption classifier. The integral (AUC) of the ROC characterizes a classifier’s performance, with a score of 1 indicating 100% true positives and 0% false negatives. Our model has an AUC of 0.9902, close to the hypothetical maximum.

3 Convolutional Heuristic for Time of Disruption

For any given shot, let the current be $I(t)$, $t \in [0, T]$. We start by noting that shots with $\max I(t) \approx 0$ are negative polarity shots and must be flipped before proceeding (Eq (4)).

$$I_{\text{raw}}(t) = \begin{cases} -I(t) & \max(I) \approx 0 \\ I(t) & \text{otherwise} \end{cases} \quad (4)$$

We then calculate a smoothed current $I_{\text{smooth}}(t)$ by applying a moving average. We compute the moving average on our dataset as a discrete convolution with window size $K = T/S$, where $S = 300$ is an experimentally determined smoothing parameter. Applying `np.convolve` on the current tensor Eq (1), we get the i th component of the smooth current $\mathbf{I}_{p,\text{smooth}}$ given by Eq 5.

$$\mathbf{I}_{p,\text{smooth}_i} = \frac{1}{K} \sum_{k=-K/2}^{K/2} I_{p,i+k} \quad (5)$$

We then define the time of disruption heuristic filter $f(t)$ as the difference between the raw current in Eq (4) and smooth current in Eq (5).

$$f(t) = I_{\text{raw}}(t) - I_{\text{smooth}}(t) \quad (6)$$

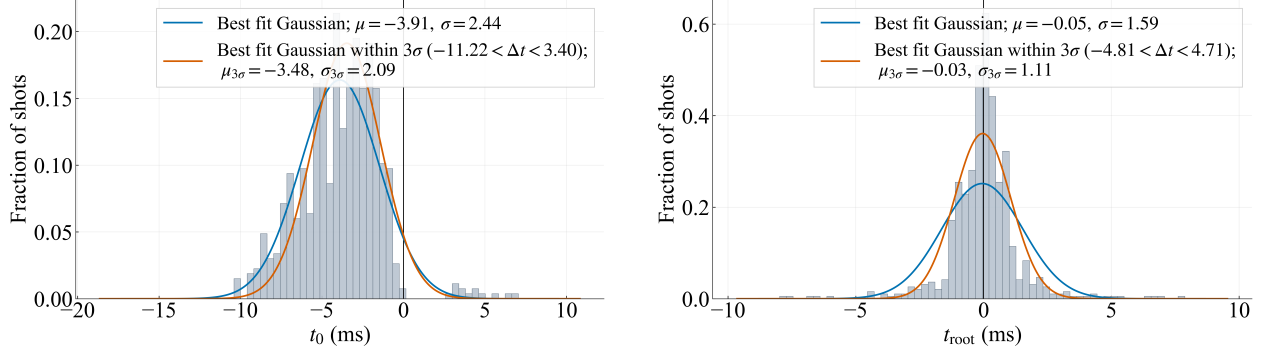


Figure 5: On the left, a histogram of the difference Δt between predicted start of disruption Eq (7) and labeled time of disruption in milliseconds. Most of our predictions are early, with a mean $\Delta t \approx -4$ ms (5a). On the right, a histogram of the difference Δt between heuristic root t_{root} Eq (8) and labeled time of disruption in milliseconds. The Gaussian is centered with $\mu \approx 0$ ms, $\sigma = 1.1$ ms (5b).

To find the start $t = t_0$ of a disruption, we simply search for the global maximum of $f(t)$. The first local minimum of $f(t)$, $t > t_0$ identifies the end of disruption t_f .

$$t_0 = \max(f(t)) \quad (7)$$

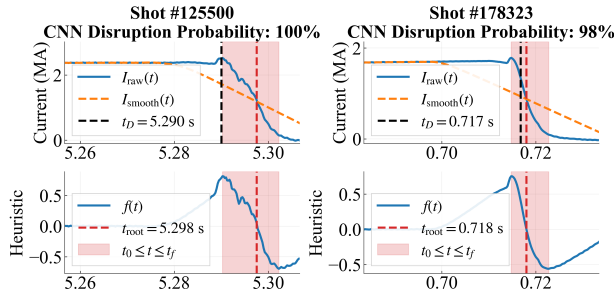


Figure 6: Two example shots (above) with their disruption heuristic $f(t)$ (below). On the left, we see shot #125500 with a DIII-D label near the disruption start Eq (7); on the right, we see shot #178323 with a DIII-D label near the disruption root Eq (8).

Comparing t_0 to the labeled disruption times $\{t_D\}$ in our test set, we find that most of our predictions are early by $\Delta t = t_0 - t_D \approx -4$ ms (Figure 5a). Therefore, we conclude that the DIII-D team labels shots mid-disruption, somewhere in the range $t_0 \leq t \leq t_f$. If we use the first root t_{root} (Eq (8)) as the predicted disruption time, we see a normalized distribution of $\Delta t = t_{\text{root}} - t_D$ centered around $\Delta t = 0$ in Figure 5b.

$$f(t_{\text{root}}) = 0, \quad t_{\text{root}} \in [t_0, t_f] \quad (8)$$

4 Conclusion

We train a multi-layer Convolutional Neural Network to classify shots from the DIII-D Tokamak Reactor as

disruptive/non-disruptive using plasma current vs. time data. We use a $F_\beta = 1.5$ performance metric to favor recall over precision and perform hyperparameter tuning over 100 trials; the trained classifier correctly identifies 99% of disruptive shots on our test holdout set, producing less than 9% false positives. We also develop a convolutional formula to identify the disruption time in disruptive shots with a median of 0 and a standard deviation of 1 ms, with an option to additionally identify the full time-span of a disruption. Our algorithm can generate automated labeling of every DIII-D shot up to the current day, and should generalize well to other Tokamak reactors, providing a much larger labeled dataset to the scientific community for developing richer models.

Acknowledgments

Our thanks to Argonne National Laboratory for providing us with the data to train our model, as well as the School of Mechanical, Industrial, and Manufacturing Engineering (MIME) at Oregon State University for providing us with funding.

Data and code availability

Code for the model, along with the best model weights and hyperparameters, are hosted at <https://github.com/starlightjason2/DLDDL.git>. We utilized Cursor [16] and Claude [17] for code development and LaTeX formatting.

References

- [1] Zhouyu Wang. “Current status of research on magnetic confinement fusion and superconducting tokamak devices”. In: *Procedia Computer Science* 228 (2023). 3rd International Conference on Machine Learning and Big Data Analytics for IoT

- Security and Privacy, pp. 163–170. ISSN: 1877-0509. DOI: <https://doi.org/10.1016/j.procs.2023.11.020>. URL: <https://www.sciencedirect.com/science/article/pii/S1877050923018434>.
- [2] Mustakimah Mohamed et al. “Global Development and Readiness of Nuclear Fusion Technology as the Alternative Source for Clean Energy Supply”. In: *Sustainability* 16.10 (2024). ISSN: 2071-1050. DOI: 10.3390/su16104089. URL: <https://www.mdpi.com/2071-1050/16/10/4089>.
- [3] A Pavone et al. “Machine learning and Bayesian inference in nuclear fusion research: an overview”. In: *Plasma Physics and Controlled Fusion* 65.5 (Apr. 2023), p. 053001. DOI: 10.1088/1361-6587/acc60f. URL: <https://doi.org/10.1088/1361-6587/acc60f>.
- [4] J. Vega et al. “Disruption prediction with artificial intelligence techniques in tokamak plasmas”. In: *Nature Physics* 18.7 (2022), pp. 741–750. ISSN: 1745-2481. DOI: 10.1038/s41567-022-01602-2. URL: <http://dx.doi.org/10.1038/s41567-022-01602-2>.
- [5] Priyanka Muruganandham et al. “Predicting Disruptions in Aditya Tokamak: A Deep Sequential Bi-LSTM Approach with Dynamic Time Window Aggregation”. In: *Fusion Science and Technology* 81.7 (2025), pp. 702–716. DOI: 10.1080/15361055.2025.2485825. eprint: <https://doi.org/10.1080/15361055.2025.2485825>. URL: <https://doi.org/10.1080/15361055.2025.2485825>.
- [6] J.X. Zhu et al. “Hybrid deep-learning architecture for general disruption prediction across multiple tokamaks”. In: *Nuclear Fusion* 61.2 (Dec. 2020), p. 026007. ISSN: 1741-4326. DOI: 10.1088/1741-4326/abc664. URL: <http://dx.doi.org/10.1088/1741-4326/abc664>.
- [7] Wei Zheng et al. “Disruption prediction for future tokamaks using parameter-based transfer learning”. In: *Communications Physics* 6.1 (2023). ISSN: 2399-3650. DOI: 10.1038/s42005-023-01296-9. URL: <http://dx.doi.org/10.1038/s42005-023-01296-9>.
- [8] Sergey Ioffe and Christian Szegedy. “Batch normalization: accelerating deep network training by reducing internal covariate shift”. In: *Proceedings of the 32nd International Conference on Machine Learning - Volume 37. ICML’15. Lille, France: JMLR.org, 2015*, pp. 448–456.
- [9] Vinod Nair and Geoffrey E. Hinton. “Rectified linear units improve restricted boltzmann machines”. In: *Proceedings of the 27th International Conference on International Conference on Machine Learning. ICML’10. Haifa, Israel: Omnipress, 2010*, pp. 807–814. ISBN: 9781605589077.
- [10] Y. Lecun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. ISSN: 0018-9219. DOI: 10.1109/5.726791. URL: <http://dx.doi.org/10.1109/5.726791>.
- [11] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. arXiv: 1912.01703 [cs.LG]. URL: <https://arxiv.org/abs/1912.01703>.
- [12] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: <https://arxiv.org/abs/1412.6980>.
- [13] Leslie N. Smith. *A disciplined approach to neural network hyper-parameters: Part 1 – learning rate, batch size, momentum, and weight decay*. 2018. arXiv: 1803.09820 [cs.LG]. URL: <https://arxiv.org/abs/1803.09820>.
- [14] Donald R. Jones, Matthias Schonlau, and William J. Welch. “Efficient Global Optimization of Expensive Black-Box Functions”. In: *Journal of Global Optimization* 13.4 (Dec. 1998), pp. 455–492. ISSN: 1573-2916. DOI: 10.1023/a:1008306431147. URL: <http://dx.doi.org/10.1023/A:1008306431147>.
- [15] C. J. Van Rijsbergen. *Information Retrieval*. 2nd. Newton, MA, USA: Butterworth-Heinemann, 1979.
- [16] Anysphere. *Cursor*. <https://www.cursor.com>. AI assistant used for code development. 2026.
- [17] Anthropic. *Claude Code (Opus 4.8)*. <https://www.anthropic.com/claude-code>. AI assistant used for code development and LaTeX formatting. 2026.